# AGENT LOOP GRAPHS

*Every loopgraph component on one page: the graph laid out from JSON, conditional edges, bounded loops with live counters, subgraphs that collapse, human interrupts, checkpoints, typed state with reducers, and a run replayed step by step.*



## 1.1 WHAT THIS IS.

A **loopgraph** is a figure for one thing: an agent graph that loops. It takes the graph as data (nodes, typed edges, the state the graph carries, and optionally one recorded run) and does the rest. Nobody places a node by hand, so a coding agent can export a graph from real code and get the figure.

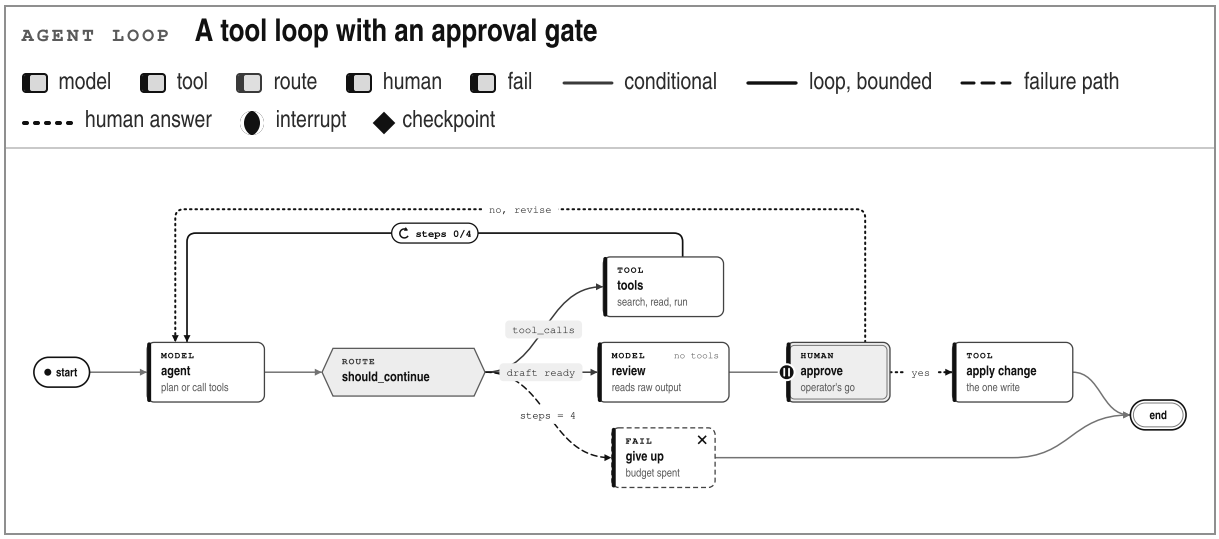
It is not the general flow diagram. Use `:::diagram` for a system map, a data path, anything without a loop counter or a state; it is hand-placed on a grid and drawn in the sheet's ink. Use `:::loop` when the point is what happens on the second pass, where the limit sits, and who has to say yes.

Table 1-1. When to use which

| Figure | For | Layout | Moves |
|---|---|---|---|
| `:::diagram` | a system map, a general flow | hand-placed on a grid | no |
| `:::trace` | a sequence of events in time | a lane and a scrubber | scrub |
| `:::loop` | an agent loop with state and bounds | computed from the graph | replay, state, checkpoints |

## 1.2 A TOOL LOOP WITH AN APPROVAL GATE.

The common shape: a model calls tools until it has a draft, a second model call with no tools reads the raw tool output, and a human approves before the one write. Press play. The run stops itself at the interrupt and waits; press resume. Open the checkpoints tab and click c3: that is the state the approval was bound to, and the state a resume would load. The *steps* badge on the loop climbs to its bound and turns amber one short of it.

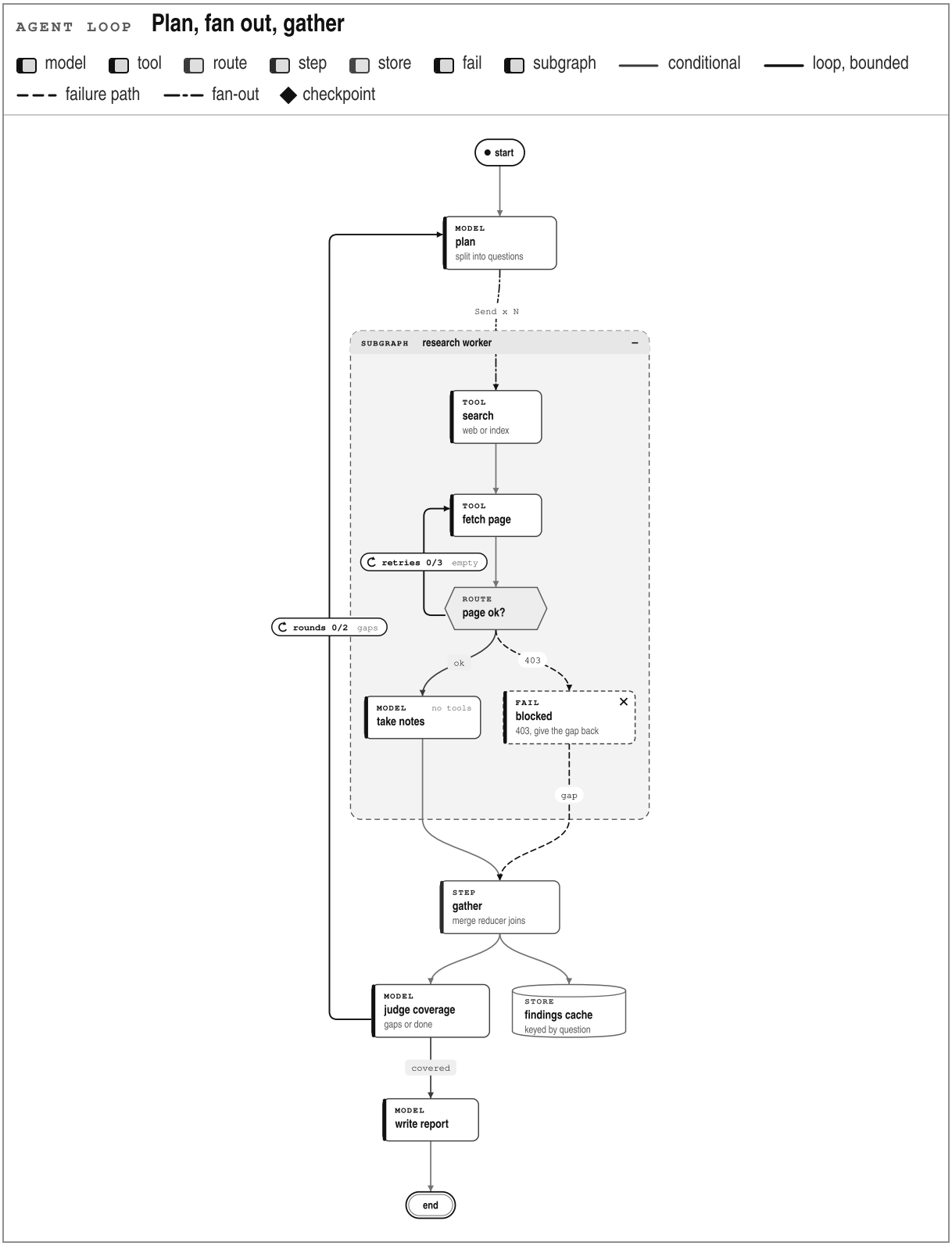



LOOP-501A

Figure 1-1. A tool loop with an approval gate. Bounded at four tool rounds; the approval names a checkpoint.

## 1.3 PLAN, FAN OUT, GATHER.

Top to bottom this time, with a subgraph. The planner fans out one worker per question (`Send`), each worker retries an empty page at most three times, a failed fetch is a dashed path that still reaches the gather, and the merge reducer keeps one finding per question. The outer loop is bounded too: two planning rounds. Click the subgraph's title bar to collapse it and see the outer loop alone.

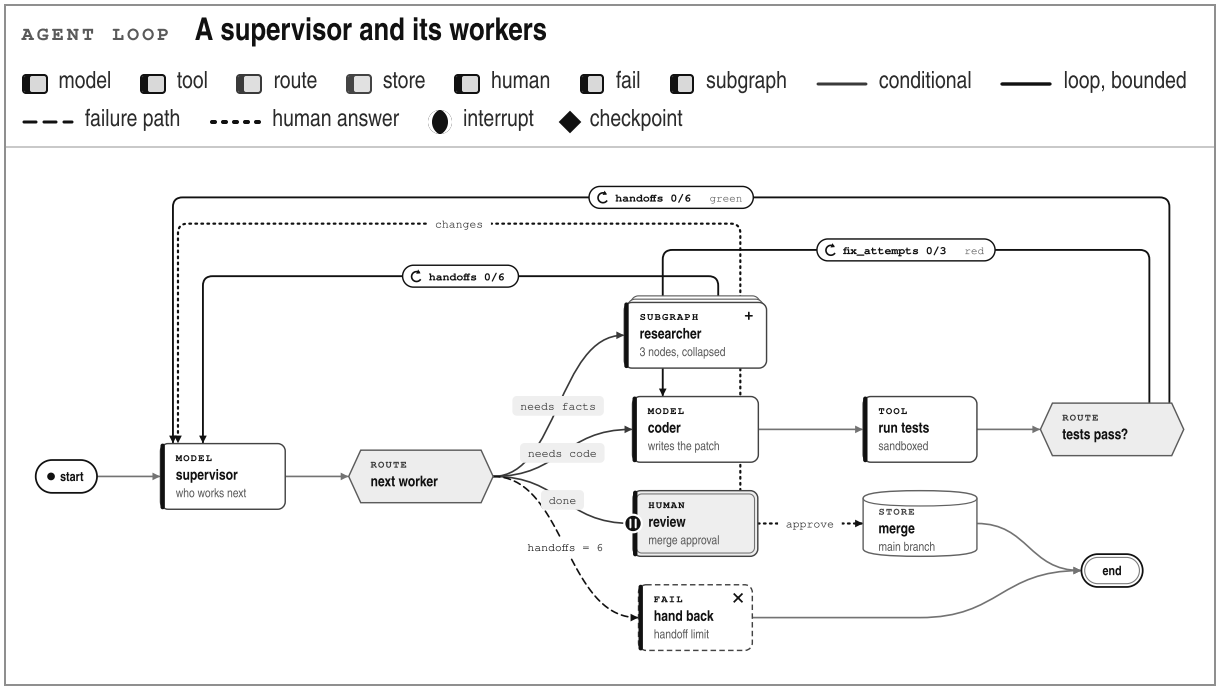



LOOP-502A

Figure 1-2. Plan, fan out, gather. Two bounded loops, one inside the other, a fan-out and a merge reducer.

## 1.4 A SUPERVISOR AND ITS WORKERS.

A supervisor routes to workers and gets the floor back after each. The researcher is a subgraph shown collapsed: the run lights it as one node. Expand it with a click. Two loop edges share the *handoffs* counter, so the bound is on the supervisor, not on any one worker; the inner test-fix loop has its own. The reviewer's "changes" is a human edge back into the loop, drawn dotted.




LOOP-503A

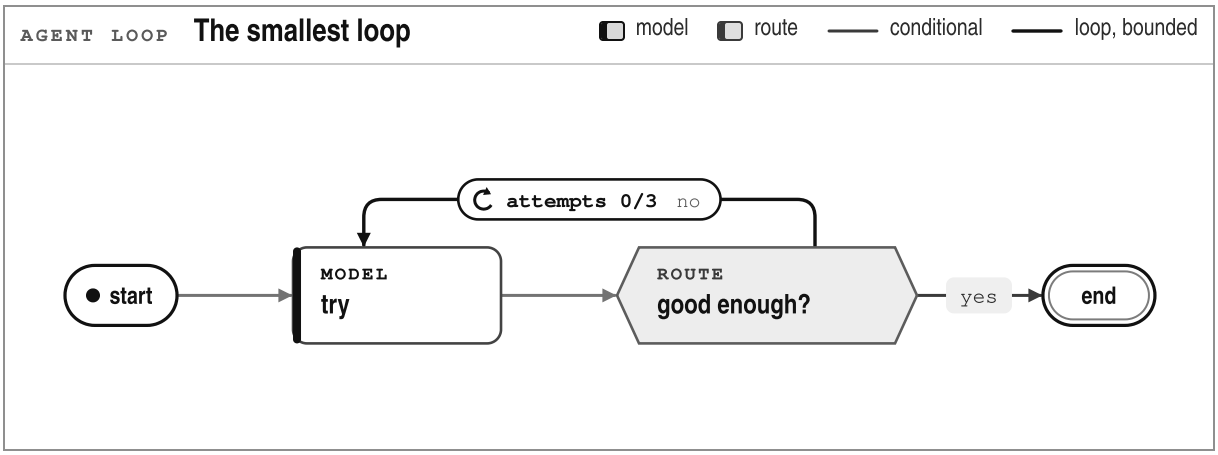
Figure 1-3. A supervisor and its workers. A collapsed subgraph, a shared handoff bound, a fix loop inside it.

## 1.5 THE SMALLEST LOOP.

Four nodes, one bounded loop, a six-step run. This is the whole of the data; everything else on this page is the same shape with more in it.

```
{
  "id": "minimal",
  "title": "The smallest loop",
  "state": {"attempts": {"reducer": "sum"}, "answer": {"reducer": "replace"}},
  "nodes": [
    {"id": "start", "kind": "start"},
    {"id": "try", "kind": "llm", "label": "try", "writes": ["answer", "attempts"]},
    {"id": "ok", "kind": "router", "label": "good enough?"},
    {"id": "end", "kind": "end"}
  ],
  "edges": [
    {"from": "start", "to": "try"},
    {"from": "try", "to": "ok"},
    {"from": "ok", "to": "try", "kind": "loop", "max": 3, "counter": "attempts", "when":
"no"},
    {"from": "ok", "to": "end", "kind": "cond", "when": "yes"}
  ],
  "run": [
    {"node": "start"},
    {"node": "try", "writes": {"answer": "draft 1", "attempts": 1}},
    {"node": "ok", "note": "Not yet."},
    {"node": "try", "via": "ok", "writes": {"answer": "draft 2", "attempts": 1}},
    {"node": "ok"},
    {"node": "end", "via": "ok"}
  ]
}
```




LOOP-504A

Figure 1-4. The smallest loop.

## 1.6 READING THE FIGURE.

Table 1-2. What each mark means

| Mark | Means |
|---|---|
| a card with a coloured stripe | a node; the stripe and the tag say its kind (model, tool, step, store) |
| a hexagon | a router: a pure function of state that picks the next edge |
| a double border and a pause mark on the edge it enters by | a human interrupt (before), or on the edge it leaves by (after) |
| a stack of cards | a collapsed subgraph; a dashed box with a title bar is the same subgraph expanded |
| a chip on an edge | the condition that takes it, or the edge's label |
| an edge lifted into a lane, with a badge | a loop's back edge; the badge is its counter against its bound |
| dashed red | a failure path |
| dotted orange | a human's answer continuing the run |
| dash-dot teal | a fan-out to parallel workers |
| a small diamond on a node | a checkpoint was saved there in the run so far |
| the graph dims, a path inks in | the run so far; the current edge flows, the current node glows |
| the meters | named limits; a recursion limit counts steps, a counter limit counts its state key or its loop |
| the chart below | the run, node by step: loops show as zig-zags, pauses and checkpoints are marked |



The state panel lists every key with its reducer. A key written at this step is lit and says how: *+1* for an append, *+n* for a sum, *set* for a replace. The writes tab is the node by key matrix: filled where a node declares it writes a key, a ring where it was only seen doing so in the run, lit where it is writing now.

## 1.7 THE DATA FORMAT.

One JSON object per figure. Only `nodes` and `edges` are required.

Table 1-3. The graph

| Key | Type | Meaning |
|---|---|---|
| `id`, `title` | string | the figure's id and the name on its title bar |
| `dir` | `LR` or `TB` | flow left to right (default) or top to bottom; long pipelines read better TB |
| `state` | object | the typed state: key to `{reducer, doc}` |
| `limits` | object | name to a number, or to `{max, of}` where `of` names the state key or loop counter it bounds |
| `groups` | array | subgraphs: `{id, label, doc, collapsed}`; one level, no nesting |
| `nodes` | array | `{id, kind, label, sub, doc, group, reads, writes, tags, interrupt}` |
| `edges` | array | `{from, to, kind, when, label, max, counter, doc}` |
| `run` | array | one recorded run, in order: `{node, via, writes, note, t, checkpoint, pause}` |

Table 1-4. Node kinds

| Kind | Draws as | For |
|---|---|---|
| `start`, `end` | a pill | the graph's entry and exit |
| `llm` | a card, slate | a model call |
| `tool` | a card, teal | a step that runs tools or commands |
| `router` | a hexagon, amber | a conditional decision |
| `human` | a double card, orange | an approval or an answer; add `interrupt: before` or `after` |
| `step` | a card, grey | plain code |
| `store` | a drum | a write to, or a read from, git, a file, a database |
| `fail` | a dashed card, red | an error exit |

Table 1-5. Edge kinds

| Kind | Draws as | Needs |
|---|---|---|
| `next` | a plain curve | nothing (the default) |
| `cond` | a curve with a chip | `when`, the condition |
| `loop` | a back edge in a lane, with a counter badge | `max` and `counter` when it is bounded |
| `fail` | dashed | a `label` saying what failed |
| `resume` | dotted | a `label`: the human's answer |
| `send` | dash-dot | a fan-out to parallel copies of a subgraph |

Table 1-6. Reducers and what a run step's writes mean

| Reducer | The key holds | A step writes |
|---|---|---|
| `replace` | the last value | the new value |
| `append` | a list | one new item |
| `sum` | a number | the increment |
| `merge` | an object | the fields to merge in |



## 1.8 RULES.

The build checks the graph before a reader sees it, and stops on any of these:

**CHECKED AT BUILD**

```
1 every edge names two nodes that exist, every
  node's group exists
2 a cond edge has a when
3 every run step names a node, and follows an
  edge from the step before it (or from via)
4 a run step writes only keys declared in state
5 node ids are unique
```

Keep labels to two to four words and conditions to a few; a chip longer than 26 characters is cut, and the full text is in the edge's hover card. Put the detail in `doc`, which the hover card shows. A graph of more than about twenty nodes wants subgraphs, collapsed by default where they are detail.